Система за извличане на информация от алгоритмични задачи на български език

Теодор Светославов Костадинов
ф.н. 5MI3400816

Извличане на Информация, зимен семестър 2025/2026

1. Избор на задача

Проблем: Всяка година от 2018 насам в курса по СДА се добавят около 30 нови задачи. Подготовката на студентите и намирането на подобни задачи става все по-трудно.

Подобряване на достъпа до материалите за студенти в СДА курса.

Цел: Изграждане на система за извличане на информация, която:

1. **Извлича** условията от HackerRank чрез уеб скрейпинг
2. **Индексира** чрез TF-IDF, BERT и хибриден модел
3. **Търси и подрежда** по прилика, оценени чрез IR метрики
4. **Предоставя** уеб интерфейс за търсене и препоръчване

2. Данни

Публичен исторически списък на всички състезания давани по СДА от 2018 насам:

Линк

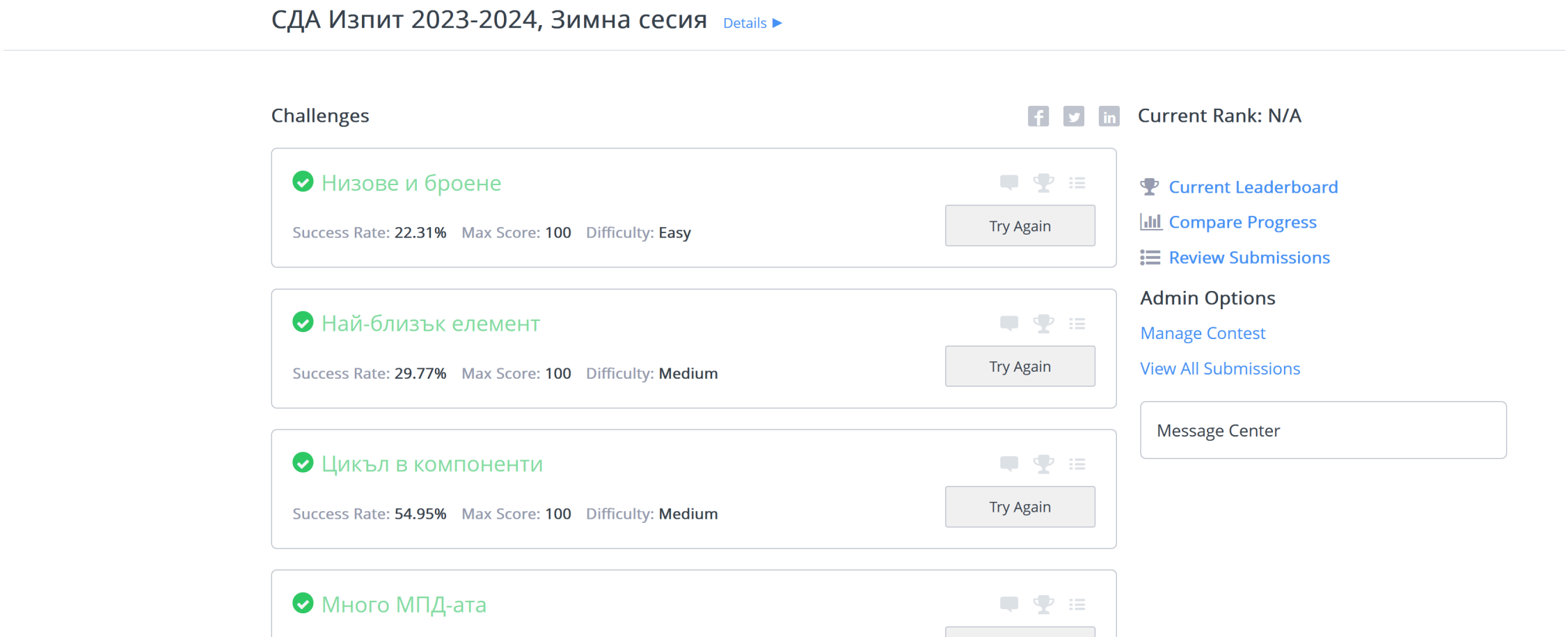
Как изглеждат данните?

Първоначалният документ конвертиран към CSV:

```
You, 3 weeks ago | 1 author (You)
year,type,link          You, 3 weeks ago • [IR] Add all links …
2018/2019,homework,www.hackerrank.com/practice-1-si
2018/2019,homework,www.hackerrank.com/si-practice-2
2018/2019,homework,www.hackerrank.com/practice-3-1
2018/2019,homework,www.hackerrank.com/si-practice-4
2018/2019,homework,www.hackerrank.com/si-practice-5
2018/2019,homework,www.hackerrank.com/sda-hw-6
2018/2019,homework,www.hackerrank.com/si-practice-7
2018/2019,homework,www.hackerrank.com/sda-hw-8
2018/2019,homework,www.hackerrank.com/sda-hw-9
2018/2019,homework,www.hackerrank.com/sda-hw-10
2018/2019,homework,www.hackerrank.com/sda-hw-11
2018/2019,exam,www.hackerrank.com/sda-test
2018/2019,exam,www.hackerrank.com/sda-ttest-2
```

Как изглеждат данните?

Състезанието https://www.hackerrank.com/contests/2023-2024-2/challenges показано чрез browser:



... и конкретна задача от състезанието например *cycles-components*...

Цикъл в компоненти

locked

Problem | Submissions | Leaderboard | Discussions

Намерете броя на свързаните компоненти в графа, в които има цикъл.

Графът ще бъде **ненасочен**. За да се установи цикъл в ненасочен граф, трябва да се спази следното правило, за да се избегне тривиалния цикъл - нямаме право да се връщаме във възела, от който сме дошли. Тоест, ако във възел 1 сме стигнали от възел 0, то нямаме право да се върнем във възел 0.

Никога няма да има повече от 1 ребро между 2 възела.

Спрямо тези правила, свързан граф с 2 върха никога няма да има цикъл.

Input Format

На първия ред ще получите едно число T - броя на заявките

За всяка заявка ще получите следното:

На първия ред ще получите две числа разделени с интервал - V и E - броя на възлите и ребрата в графа.

На следващите E на брой реда ще получите по 2 числа, разделени с интервал - $from$ и to, между кои два възела има ребро.

Constraints

$1 \leq T \leq 10$

$1 \leq V \leq 10^6$

Submissions: 111
Max Score: 100
Difficulty: Medium

Rate This Challenge:

More

Admin Options

Edit Challenge

View Submissions

```
{
  "can_be_viewed": true,
  "can_edit": null,
  "bookmarked": false,
  "dynamic": true,
  "has_started": true,
  "has_ended": false,
  "countdown_time": 0,
  "requirements_description": null,
  "max_score": 100,
  "active": true,
  "epoch_starttime": null,
  "epoch_endtime": null,
  "time_left": null,
  "factor": 100,
  "expert_solution_status": false,
  "custom_tabs": null,
  "total_count": 111,
  "solved_count": 61,
  "success_ratio": 0.5495495495495496,
  "is_editorial_available": false,
  "is_solution_unlocked": false,
  "contest_slug": "2023-2024-2",
  "topics": [],
  "user_score": 0.0,
  "track": null,
  "id": 389099,
  "slug": "cycles-components",
  "name": "Цикъл в компоненти",
  "status": null,
  "created_at": "2024-01-13T20:26:41.000Z",
  "updated_at": "2024-02-01T10:15:40.000Z",
  "kind": "code",
  "preview": "wtf",
  "category": "ai",
  "deleted": false,
  "company_id": null,
  "is_custom": false,
  "player_count": 2,
  "custom_checker_language": null,
  "checker_program": null,
  "judgebot_language": null,
  "judgebot": null,
```


Как изглеждат данните?

Във JSON format след скрейпване:

Атрибутите, които ще използваме:

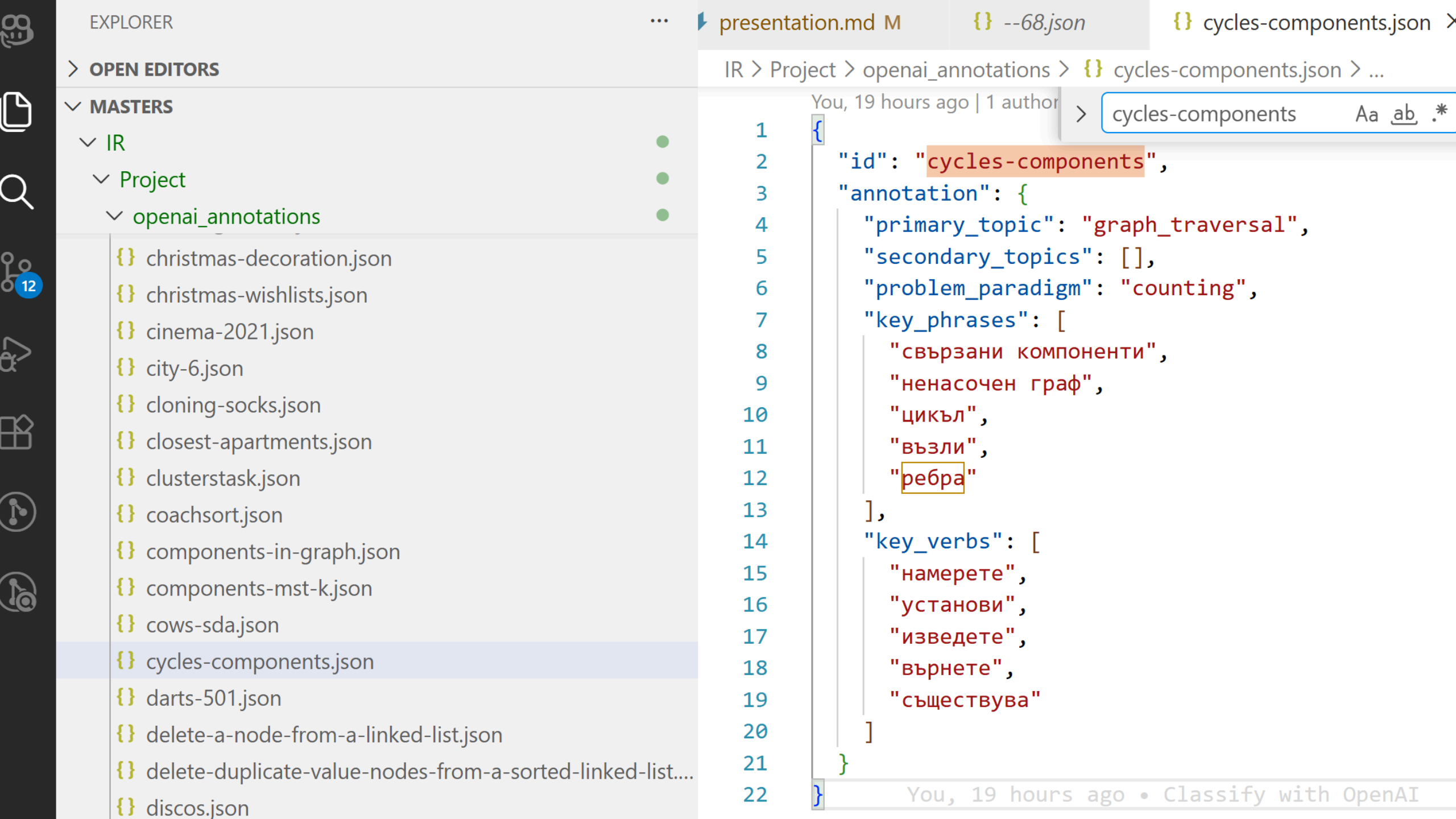
```
225     "input_format": "На първия ред ще получите едно число $T$ - броя на заявките
226     "output_format": "За всяка заявка $T_i$ изведете едно число на нов ред - бро
227     "problem_statement": "Намерете броя на свързаните компоненти в графа, в кои
228     "constraints": "$1 \\leq T \\leq 10$\n\n$1 \\leq V \\leq 10^6$\n\n$1 \\leq
229 }
```

3. Данни - Уеб скрейпинг

- Автоматизиран скрейпър с **Playwright** (нужда от автентикация)
- Извлича: условие, трудност, точки, успеваемост, тагове, състезание
- Динамично разширяване + дедупликация

Анотиране на данните

- Слабо анотиране чрез OpenAI агент
- Дефиниране на класове задачи според категориите лекции по конспект


EXPLORER
OPEN EDITORS
MASTERS
IR
Project
openai_annotations
christmas-decoration.json
christmas-wishlists.json
cinema-2021.json
city-6.json
cloning-socks.json
closest-apartments.json
clusterstask.json
coachsort.json
components-in-graph.json
components-mst-k.json
cows-sda.json
cycles-components.json
darts-501.json
delete-a-node-from-a-linked-list.json
delete-duplicate-value-nodes-from-a-sorted-linked-list....
discos.json
presentation.md M
--68.json
cycles-components.json
IR > Project > openai_annotations > cycles-components.json > ...
You, 19 hours ago | 1 author
cycles-components
{
"id": "cycles-components",
"annotation": {
"primary_topic": "graph_traversal",
"secondary_topics": [],
"problem_paradigm": "counting",
"key_phrases": [
"свързани компоненти",
"ненасочен граф",
"цикъл",
"възли",
"ребра"
],
"key_verbs": [
"намерете",
"установи",
"изведете",
"върнете",
"съществува"
]
}
}
You, 19 hours ago • Classify with OpenAI

Как изглеждат данните? (обобщение)

- 329 текстови файла
- Съдържат условието на всяка задача в суров текстов формат
- Категория на задачата (*label*)

```
Total files: 329

=== Primary Topic Counts ===
  dynamic_programming                      68
  graph_traversal                          52
  sorting                                  44
  binary_search_tree                       38
  hash_table                               32
  linked_list                              25
  stack_queue                              23
  shortest_path                            20
  searching                                 8
  minimum_spanning_tree                     8
  priority_queue                            8
  eulerian_hamiltonian                      1
```

4. Предобработка на текста

Стъпка	Описание
Нормализация	Премахване на HTML, Markdown, LaTeX, малки букви
Извличане на секции	Условие + входен формат + изходен формат
Токенизация и лематизация	**CLASSLA** - специализиран за южнославянски езици
Стоп-думи	**stopwordsiso** за български

5. Модели за извличане на информация

Модел I: TF-IDF

- Лематизиран текст към TF-IDF вектори (униграми + биграми)
- Косинусова мярка за прилика

Предимство: Силен при категории с характерна лексика

Модел II: BERT

- Многоезичен *Transformer* за изречения
- Косинусова мярка за прилика

Предимство: Семантично разбиране отвъд лексиката

Модел III: Хибриден

Линейна интерполация на TF-IDF и BERT:

$$score = (1 - \alpha) \times tfidf_sim + \alpha \times bert_sim$$

- α вариране на параметъра от 0.0 до 1.0 със стъпка 0.1
- **Оптимално: α = 0.4** → 60% TF-IDF + 40% BERT

Предимство: Комбинира лексикално и семантично търсене

6. Методика на оценка

- ***Leave-one-out***: за всяка от 329 задачи
- Проверка дали принадлежат към **същата категория**

Метрика	Описание
P@K	Дял на релевантните сред Top-K
MRR	Средна реципрочна позиция на първия релевантен
MAP@5	Средна точност на подреждането до позиция 5

7. Резултати

Допълнителни NLP експерименти

Експеримент	Метрика	Стойност
PMI класификатор	Accuracy	0.333
PMI със слети класове	Top-1	0.226
PMI със слети класове	Top-5	0.903
Извличане на слотове	Accuracy	0.307

*Подходите чрез правила са **недостатъчни***

Сравнение на моделите за извличане

Модел	P@1	P@3	P@5	MRR	MAP@5
TF-IDF (лематизиран)	0.511	0.441	0.423	0.614	0.587
BERT	0.465	0.424	0.403	0.590	0.560
Хибриден (α=0.4)	**0.535**	**0.453**	**0.423**	**0.638**	**0.605**

Хибридният модел е най-добър по всички метрики.

Анализ по категории (P@1)

Категория	TF-IDF	BERT	Хибриден	Брой
binary_search_tree	0.842	0.763	**0.921**	38
linked_list	**0.840**	0.800	**0.840**	25
graph_traversal	**0.673**	0.596	0.654	52
shortest_path	**0.600**	0.400	**0.650**	20
sorting	**0.500**	0.341	0.477	44
dynamic_programming	0.368	0.353	**0.441**	68
hash_table	0.219	**0.281**	0.219	32
minimum_spanning_tree	0.250	**0.625**	0.375	8

Анализ на резултатите

- **TF-IDF** превъзхожда при лексикално богати категории
  BST, linked_list → P@1 > 84%
- **BERT** е по-добър при семантично сходни задачи
  hash_table, minimum_spanning_tree
- **Хибридът** подобрява 4/9 категории спрямо TF-IDF
  BST: 0.842 → 0.921, DP: 0.368 → 0.441
- **Трудни категории**: dynamicprogramming, hashtable
  Определят се от метода на решение, не от лексиката

8. Уеб интерфейс (Streamlit)

Раздел	Функционалност
Search	Търсене по свободен текст, настройка на α
Browse	Сортируема таблица с филтри по тема, трудност, състезание
Topics	Статистика и визуализация по теми, разпределение на трудност
Practice	Препоръчване на задачи на базата на решени

Демонстрация

9. Използвани технологии

Компонент	Технология
Уеб скрейпинг	Playwright
TF-IDF индексиране	scikit-learn (TfidfVectorizer)
BERT ембединги	sentence-transformers
NLP за български	CLASSLA
Стоп-думи	stopwordsiso
Уеб интерфейс	Streamlit
Анотиране	OpenAI API

10. Заключение

- Пълна IR система: скрейпинг → индексиране → търсене → уеб интерфейс
- **329 задачи** извлечени, **анотирани** с 14 категории
- Хибриден модел (60% TF-IDF + 40% BERT): **P@1 = 53.5%, MRR = 0.638**
- Допълнителни NLP експерименти потвърждават нуждата от векторни модели

11. Бъдещо развитие

- **Разширяване на заявки**
- **Learning to Rank** - модел за оптимално комбиниране на множество сигнали
- **Подобряване на анотациите** - ръчна проверка на OpenAI етикетите
- **Публично разгръщане** - предварително изчислени индекси и публикуване в облака

Време за въпроси